

Problema de Coloração de Grafos

Dalton Solano dos Reis*

Andre Tavares da Silva†

2026

Resumo

Este artigo analisa o Problema de Coloração de Grafos na formulação de decisão para três cores, comparando uma solução por força bruta com uma versão baseada em redução para SAT, conversão para 3-CNF-SAT e resolução por DPLL com propagação unitária. O estudo parte da caracterização teórica do problema, de suas aplicações e de sua relação com problemas NP-Completo, e implementa as duas abordagens em um mesmo ambiente experimental. A versão original enumera todas as atribuições possíveis de cores e valida cada candidata diretamente no grafo. A versão otimizada codifica restrições de unicidade de cor e incompatibilidade entre vértices adjacentes como cláusulas booleanas, explorando simplificação lógica antes das ramificações do DPLL. Nos experimentos registrados, a força bruta apresentou crescimento coerente com $O(3^{|V|})$, enquanto a versão otimizada reduziu o número de decisões e o tempo médio observado, embora com maior consumo de memória devido ao armazenamento da fórmula CNF. Os resultados indicam que reduções polinomiais para SAT podem melhorar o desempenho prático em instâncias pequenas e médias quando a propagação elimina grandes partes do espaço de busca, mas não removem o pior caso exponencial inerente à formulação de decisão do problema.

Palavras-chave: Coloração de grafos. SAT. 3-CNF-SAT. DPLL. Complexidade de algoritmos.

1 Fundamentação Teórica

O Problema de Coloração de Grafos consiste em atribuir cores aos vértices de um grafo $G = (V, E)$ de modo que vértices adjacentes recebam cores distintas. Na versão de decisão adotada neste trabalho, recebe-se também um inteiro k e pergunta-se se existe uma função $c : V \rightarrow \{1, \dots, k\}$ tal que $c(u) \neq c(v)$ para toda aresta $(u, v) \in E$. A versão de otimização procura o menor número de cores, chamado número cromático, mas a formulação de decisão é suficiente para discutir tratabilidade, certificação e reduções polinomiais, pois uma coloração candidata pode ser verificada em tempo proporcional ao número de vértices e arestas (Cormen *et al.*, 2024; Dasgupta; Papadimitriou; Vazirani, 2009).

*PPGCAP - UDESC, e-mail: dalton@furb.br

†PPGCAP - UDESC, e-mail: andre.silva@udesc.br

A dificuldade computacional surge porque o espaço de busca cresce exponencialmente com $|V|$. Para k cores, há $k^{|V|}$ atribuições possíveis antes de considerar as restrições impostas pelas arestas. Um certificado, entretanto, é simples: basta fornecer uma cor para cada vértice e verificar, para cada aresta, se os extremos diferem. Essa assimetria entre busca e verificação aproxima o problema da discussão clássica entre problemas em P, NP e NP-Completo apresentada nos materiais da disciplina (Silva, 2026a). Golovach *et al.* (2016) mostram que a complexidade de variantes de coloração permanece sensível a restrições estruturais do grafo, o que explica por que classes particulares admitem algoritmos eficientes, enquanto a formulação geral continua exigindo técnicas combinatórias ou reduções.

As aplicações decorrem da interpretação de cores como recursos mutuamente incompatíveis. Em escalonamento, vértices podem representar tarefas, exames ou transmissões, enquanto arestas indicam conflitos que impedem execução simultânea. Em alocação de registradores, variáveis cujos intervalos de vida se sobrepõem não podem ocupar o mesmo registrador físico; a coloração do grafo de interferência modela essa restrição. Também há usos em atribuição de frequências, particionamento de mapas, organização de horários e problemas de separação de entidades conflitantes (Lewis, 2026; Artacho; Campoy, 2016). A utilidade do modelo não elimina a dificuldade algorítmica: quando o grafo é denso ou contém muitos ciclos e cliques, pequenas escolhas locais podem bloquear colorações futuras, ampliando a necessidade de retrocesso ou propagação de restrições.

A versão otimizada estudada neste artigo usa uma cadeia de reduções clássica. Primeiro, a coloração é transformada em uma instância SAT. Para cada par formado por vértice v e cor i , cria-se uma variável booleana $x_{v,i}$, verdadeira quando v recebe a cor i . As cláusulas impõem três propriedades: cada vértice possui pelo menos uma cor; cada vértice possui no máximo uma cor; e, para cada aresta (u, v) , os vértices u e v não compartilham a mesma cor. Essa codificação preserva satisfazibilidade: uma atribuição booleana satisfatória corresponde a uma coloração válida, e uma coloração válida induz uma atribuição que satisfaz todas as cláusulas (Reis, 2026).

A segunda etapa converte a fórmula para 3-CNF-SAT. Uma fórmula está em CNF quando é uma conjunção de cláusulas, cada uma composta por uma disjunção de literais. Em 3-CNF, cada cláusula possui três literais. O material de reduções da disciplina apresenta 3-CNF-SAT como uma forma central para provas de NP-Completo, pois SAT pode ser reduzido polinomialmente a 3-CNF-SAT (Silva, 2026b). Na implementação avaliada, como $k = 3$, as cláusulas geradas possuem tamanho máximo três; cláusulas unitárias ou binárias são ajustadas por repetição de literais, preservando a satisfazibilidade sem introduzir novas variáveis auxiliares. Essa escolha é adequada para uma implementação didática, mas não substitui transformações gerais, como codificações por variáveis auxiliares, quando cláusulas maiores precisam ser tratadas.

A terceira etapa resolve a fórmula por DPLL. O algoritmo DPLL realiza busca em profundidade sobre atribuições booleanas, mas aplica simplificações antes de ramificar: cláusulas satisfeitas são removidas, literais falsos são descartados e cláusulas unitárias forçam atribuições necessárias. A propagação unitária reduz o espaço explorado porque uma única decisão pode determinar várias variáveis por consequência lógica. Ainda assim, o pior caso permanece exponencial, pois a redução para SAT reorganiza o problema, mas não altera a natureza NP-Completo da formulação geral (Cormen, 2014; Cormen *et al.*, 2024). Essa fundamentação permite comparar, nas próximas seções, uma enumeração direta de colorações com uma abordagem que explicita restrições e explora inferência booleana antes da busca.

2 Apresentação da Solução Original do algoritmo

A solução original implementada no Grupo 2 do notebook resolve a coloração por enumeração exaustiva. O algoritmo recebe um grafo representado por listas de adjacência e um número fixo de cores, definido no estudo como $k = 3$. Em seguida, gera todas as tuplas de tamanho $|V|$ sobre o conjunto de cores e interpreta cada tupla como uma coloração candidata. A função de validação percorre as arestas e rejeita a candidata quando encontra dois vértices adjacentes com a mesma cor. A execução termina no primeiro certificado válido ou após esgotar todas as combinações (Reis, 2026).

Listagem 1: Núcleo da solução original por força bruta

```
def colorir_forca_bruta_original(grafo, numero_cores):
    vertices = list(grafo.keys())
    tentativas = 0

    for cores_candidatas in product(range(numero_cores),
                                     repeat=len(vertices)):
        tentativas += 1
        coloracao = dict(zip(vertices, cores_candidatas))
        if coloracao_valida(grafo, coloracao):
            return True, coloracao, tentativas

    return False, None, tentativas
```

A Listagem 1 evidencia a ausência de poda, ordenação de vértices ou propagação de restrições. Essa escolha torna o comportamento do algoritmo transparente: a complexidade decorre diretamente da cardinalidade do produto cartesiano. Para $n = |V|$ e $m = |E|$, há k^n candidatas, e cada validação custa $O(n + m)$ quando a estrutura de verificação percorre a coloração e as adjacências. Assim, o custo temporal no pior caso é $O(k^n \cdot (n + m))$; para $k = 3$, obtém-se $O(3^n \cdot (n + m))$. O consumo adicional de memória é $O(n)$, pois o algoritmo mantém apenas a tupla corrente, o dicionário de coloração e variáveis auxiliares de controle.

O fluxo da solução original pode ser resumido pelo diagrama Mermaid reproduzido na Listagem 2, extraído da organização conceitual do notebook. A decisão metodológica de apresentar o diagrama como listagem preserva a rastreabilidade do artefato original sem introduzir dependência externa de renderização no processo LaTeX.

Listagem 2: Fluxo da solução original em Mermaid

```
flowchart LR
    A["Grafo G=(V,E)"] --> B["Solução original: força bruta"]
    B --> I["Grafo colorido"]
    I --> F["Comparação de tempo e memória"]
```

No grafo de exemplo com seis vértices, a solução original encontrou uma coloração válida após 141 tentativas, com tempo registrado de 0,001601 s e pico de memória de 6,12 KiB. Em grafos completos K_n , usados no Grupo 4 para criar instâncias difíceis quando $n > 3$, o algoritmo esgotou 3^n combinações nas instâncias não 3-coloríveis. Essa característica é relevante: instâncias sem solução podem ser mais custosas do que instâncias satisfazíveis quando uma solução aparece cedo na ordem de enumeração. O método, portanto, é apropriado como linha de base experimental e como implementação de referência, mas

não escala para grafos maiores porque não aprende com rejeições anteriores nem antecipa conflitos locais.

A principal vantagem da solução original é sua simplicidade operacional. O algoritmo não depende de transformações intermediárias, sua correção é imediata pela enumeração completa do espaço de soluções, e o baixo consumo de memória favorece instâncias pequenas. A limitação é estrutural: qualquer aumento em $|V|$ multiplica o espaço de busca por k , e a validação de uma candidata inválida não impede que candidatas equivalentes, contendo o mesmo conflito parcial, sejam avaliadas posteriormente. Essa limitação motiva a versão otimizada, que representa as restrições em uma fórmula booleana para permitir simplificações antes da exploração exaustiva.

3 Apresentação da Versão Otimizada do algoritmo

A versão otimizada, implementada no Grupo 3 do notebook, substitui a enumeração direta de colorações por uma formulação lógica. O algoritmo reduz a coloração de grafos para SAT, converte a fórmula para 3-CNF-SAT e aplica DPLL com propagação unitária. A otimização não consiste em alterar a resposta do problema, mas em reorganizar o espaço de busca para que conflitos sejam detectados por inferência booleana antes que combinações completas de cores sejam construídas (Reis, 2026).

Listagem 3: Fluxo da versão otimizada em Mermaid

flowchart TD

```

A["Coloração de Grafos"] --> B["Variáveis booleanas x_v_c"]
B --> C["Cláusulas: pelo menos uma cor"]
B --> D["Cláusulas: no máximo uma cor"]
B --> E["Cláusulas: arestas com cores diferentes"]
C --> F["CNF"]
D --> F
E --> F
F --> G["3-CNF-SAT"]
G --> H["DPLL"]

```

A codificação cria uma variável $x_{v,c}$ para cada vértice v e cor c . A cláusula $(x_{v,1} \vee x_{v,2} \vee x_{v,3})$ impõe que v receba ao menos uma cor. As cláusulas binárias $(\neg x_{v,a} \vee \neg x_{v,b})$, para pares distintos de cores, impedem que o mesmo vértice receba duas cores simultaneamente. Para cada aresta (u, v) e cor c , a cláusula $(\neg x_{u,c} \vee \neg x_{v,c})$ impede que os extremos compartilhem a cor c . Essa construção é polinomial: para $n = |V|$, $m = |E|$ e k cores, cria $O(nk)$ variáveis e $O(nk^2 + mk)$ cláusulas. Para $k = 3$, isso corresponde a $3n$ variáveis e $4n + 3m$ cláusulas antes das simplificações de execução.

Listagem 4: Geração das cláusulas CNF para coloração

```

for vertice in sorted(grafo):
    pelo_menos_uma_cor = tuple(
        variavel_por_par[(vertice, cor)]
        for cor in range(numero_cores)
    )
    formula.append(pelo_menos_uma_cor)

for cor_a in range(numero_cores):

```

```

        for cor_b in range(cor_a + 1, numero_cores):
            formula.append((
                -variavel_por_par[(vertice, cor_a)],
                -variavel_por_par[(vertice, cor_b)]
            ))

for u, v in listar_arestas(grafo):
    for cor in range(numero_cores):
        formula.append((
            -variavel_por_par[(u, cor)],
            -variavel_por_par[(v, cor)]
        ))

```

A conversão para 3-CNF-SAT preserva a satisfazibilidade por repetição de literais em cláusulas de tamanho um ou dois. Por exemplo, uma cláusula $(a \vee b)$ é representada como $(a \vee b \vee b)$. Essa estratégia é correta para a implementação analisada porque a redução para três cores não gera cláusulas com mais de três literais. Em uma formulação com $k > 3$, a cláusula “pelo menos uma cor” teria tamanho k e exigiria transformação geral, usualmente com variáveis auxiliares, para manter crescimento polinomial (Silva, 2026b). Portanto, a otimização avaliada deve ser lida como uma solução especializada para a decisão de 3-coloração, não como conversor 3-CNF universal.

O DPLL usado na solução otimizada aplica propagação unitária antes de escolher uma variável para ramificação. A propagação percorre a fórmula simplificada e identifica cláusulas que, após atribuições anteriores, contêm apenas um literal efetivo; esse literal precisa ser verdadeiro para evitar contradição. A escolha de variável usa uma heurística simples de frequência: seleciona a variável não atribuída que aparece mais vezes nas cláusulas remanescentes. Essa escolha tende a antecipar decisões com maior impacto porque uma variável frequente participa de mais restrições, embora não garanta redução assintótica no pior caso.

Listagem 5: Estrutura do DPLL com propagação unitária

```

formula_atual, atribuicao_atual = propagar_unitarias(formula,
                                                    atribuicao)

if formula_atual is None:
    return False, atribuicao_atual, estatisticas
if not formula_atual:
    return True, atribuicao_atual, estatisticas

variavel = escolher_variavel(formula_atual, atribuicao_atual)

for valor in (True, False):
    estatisticas["decisoas"] += 1
    nova_atribuicao = dict(atribuicao_atual)
    nova_atribuicao[variavel] = valor
    satisfazivel, solucao, estatisticas = resolver_dpll(
        formula_atual, nova_atribuicao, estatisticas
    )
    if satisfazivel:

```

`return True, solucao, estatisticas`

Após encontrar uma atribuição satisfatória, o algoritmo reconstrói a coloração lendo as variáveis verdadeiras associadas aos pares (v, c) e valida novamente a solução no grafo original. Essa verificação final reduz o risco de erro de implementação na tradução entre modelos, pois uma atribuição booleana pode conter variáveis auxiliares ou combinações irrelevantes para a coloração reconstruída. No exemplo com seis vértices, a solução otimizada encontrou coloração válida com duas decisões, tempo de 0,000464 s e pico de memória de 14,97 KiB. O ganho de tempo decorre da eliminação de atribuições inconsistentes ainda parciais; o custo é o armazenamento da fórmula e das estruturas de mapeamento. A próxima seção quantifica esse compromisso entre processamento, memória e número de decisões.

4 Comparação entre Original e Otimizada

A comparação entre as duas versões deve separar complexidade assintótica e desempenho observado. A solução original opera diretamente sobre colorações candidatas; a versão otimizada constrói uma instância booleana e executa busca sobre variáveis SAT. Ambas enfrentam um problema exponencial no pior caso, mas exploram espaços de busca com granularidades diferentes. Na força bruta, cada tentativa corresponde a uma coloração completa. No DPLL, uma decisão booleana pode determinar várias consequências por propagação unitária, o que reduz o número de ramificações em instâncias nas quais as restrições produzem cláusulas unitárias cedo (Reis, 2026).

Tabela 1: Comparação assintótica das versões analisadas

Versão	Tempo	Memória adicional
Original por força bruta	$O(k^{ V } \cdot (V + E))$; para $k = 3$, $O(3^{ V } \cdot (V + E))$	$O(V)$ para a coloração candidata
Otimizada por SAT/DPLL	Redução em $O(V k^2 + E k)$ e DPLL exponencial no pior caso, limitado por $O(2^{ V k})$ em termos das variáveis booleanas	$O(V k + V k^2 + E k)$ para mapeamentos, fórmula e atribuição

A Tabela 1 mostra que a otimização não transforma o problema em polinomial. O benefício está no fator prático de busca: a codificação SAT explicita restrições que a força bruta só detecta após montar uma coloração inteira. Para $k = 3$, a fórmula possui $3|V|$ variáveis booleanas. O número de cláusulas cresce linearmente em $|E|$ para as restrições entre adjacentes e linearmente em $|V|$ para as restrições internas de cada vértice, uma vez que k é constante. Essa estrutura aumenta a memória, mas fornece ao DPLL informação suficiente para simplificar a fórmula a cada atribuição.

Os dados da Tabela 2 foram coletados com uma repetição por tamanho, usando grafos completos de três a nove vértices. Para $n > 3$, esses grafos não são 3-coloríveis, pois contêm clique de tamanho maior que três; por isso, a força bruta precisa esgotar todas as combinações. O número de tentativas acompanha exatamente $3^{|V|}$ nas instâncias sem solução. A versão otimizada manteve dez decisões de DPLL de K_4 a K_9 , porque as cláusulas de incompatibilidade em grafos completos produzem contradições rapidamente depois de poucas atribuições. O tempo otimizado cresce de modo mais suave no intervalo avaliado, embora ainda inclua o custo de construir e simplificar a fórmula.

Tabela 2: Resultados observados em grafos completos K_n com três cores

$ V $	$ E $	Tentativas	Decisões	Tempo original (s)	Tempo otimizado (s)
3	3	6	2	0,000045	0,000178
4	6	81	10	0,000412	0,000722
5	10	243	10	0,001454	0,001319
6	15	729	10	0,004996	0,001583
7	21	2187	10	0,017849	0,002106
8	28	6561	10	0,057489	0,002886
9	36	19683	10	0,203038	0,003302

O uso de memória apresenta comportamento oposto ao tempo. No conjunto de grafos completos, a média de memória da força bruta foi 2,35 KiB, enquanto a versão otimizada atingiu média de 19,48 KiB. A diferença é esperada: a força bruta descarta cada candidata após a validação, ao passo que o método SAT mantém mapeamentos entre pares (v, c) e variáveis, listas de cláusulas, atribuições parciais e cópias simplificadas durante a recursão. A decisão de otimizar processamento por representação lógica, portanto, desloca parte do custo para memória.

Os tempos médios registrados foram 0,040755 s para a solução original e 0,001728 s para a otimizada. Esses valores não devem ser generalizados como fator universal de aceleração, pois dependem da ordem de enumeração, da densidade dos grafos, da implementação em Python, do coletor de memória e do conjunto de instâncias. Ainda assim, eles evidenciam uma tendência coerente com a análise teórica: quando a instância contém muitas restrições, a propagação unitária e a escolha de variáveis frequentes conseguem detectar inviabilidade antes da enumeração completa. Em grafos esparsos ou em instâncias satisfazíveis cuja primeira coloração válida aparece cedo, a força bruta pode competir em tempo absoluto por evitar o custo fixo da redução.

5 Considerações Finais

Este trabalho comparou duas estratégias para o Problema de Coloração de Grafos com três cores: uma solução original por força bruta e uma versão otimizada baseada em redução para SAT, conversão para 3-CNF-SAT e DPLL com propagação unitária. A análise confirma que a enumeração direta tem valor como linha de base, pois sua correção decorre da cobertura completa do espaço de colorações e sua memória adicional é reduzida. O custo exponencial, entretanto, aparece de forma imediata quando a instância não admite coloração, já que todas as $3^{|V|}$ candidatas precisam ser rejeitadas.

A versão otimizada foi efetiva nas instâncias avaliadas porque a codificação booleana tornou explícitas as restrições do problema. As cláusulas de unicidade de cor e de incompatibilidade entre vértices adjacentes permitem que o DPLL derive consequências antes de completar uma coloração. Em grafos completos não 3-coloríveis, essa representação produziu contradições com poucas decisões, reduzindo o tempo de execução em relação à força bruta. A melhora, porém, não equivale a uma mudança de classe de complexidade: 3-coloração e SAT permanecem problemas de decisão exponenciais no pior caso, e Silva (2026b) situam 3-CNF-SAT entre as reduções centrais para NP-Completo.

O principal custo da otimização foi o aumento de memória. A solução SAT/DPLL precisa armazenar variáveis, cláusulas, mapeamentos e atribuições parciais; por isso, a média de memória observada superou a da força bruta. Esse trade-off é aceitável quando

o tempo de busca domina o custo total, especialmente em grafos densos ou em instâncias insatisfazíveis. Em grafos muito pequenos, esparsos ou com coloração encontrada nas primeiras tentativas, o custo fixo da redução pode neutralizar o ganho obtido pela propagação. A implementação também usa uma conversão 3-CNF adequada ao caso $k = 3$, mas limitada para valores maiores de k , nos quais cláusulas longas exigiriam transformação geral com variáveis auxiliares.

As ameaças à validade concentram-se no tamanho e na diversidade das instâncias. Os experimentos usaram grafos completos entre três e nove vértices e uma repetição por tamanho, o que favorece a observação de conflitos fortes e não representa todos os padrões estruturais encontrados em aplicações reais. Além disso, as medições foram feitas em Python, com `tracemalloc` para memória e tempos muito pequenos, sensíveis a variações do ambiente de execução. Estudos posteriores devem incluir grafos aleatórios com diferentes densidades, grafos planares, instâncias satisfazíveis e insatisfazíveis balanceadas, múltiplas repetições e comparação com heurísticas clássicas de coloração, como ordenação por grau e retrocesso com poda.

Como recomendação prática, a solução otimizada é mais vantajosa quando há densidade de restrições suficiente para alimentar a propagação unitária ou quando a inexistência de coloração precisa ser demonstrada sem enumerar colorações completas. A força bruta permanece útil para validação, ensino e testes pequenos, nos quais simplicidade e baixo uso de memória são preferíveis à infraestrutura de redução. A comparação mostra que reduções polinomiais para SAT são ferramentas relevantes não apenas para provas de complexidade, mas também para engenharia de algoritmos: elas deslocam o problema para uma representação na qual inferências locais podem reduzir substancialmente a busca, desde que o custo de memória e as limitações da codificação sejam controlados.

Referências

ARTACHO, Francisco J. Aragón; CAMPOY, Rubén. **Solving graph coloring problems with the Douglas-Rachford algorithm**. 15 dez. 2016. DOI: 10.48550/arXiv.1612.05026. arXiv: 1612.05026 [math.OC]. Disponível em: <http://arxiv.org/abs/1612.05026>. Acesso em: 19 jun. 2026. Pré-publicado.

CORMEN, Thomas H. **Desmistificando algoritmos**. Rio de Janeiro: Elsevier, 2014.

CORMEN, Thomas H. *et al.* **Algoritmos: teoria e prática**. 4. ed. Rio de Janeiro: LTC, 2024.

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H.; VAZIRANI, Umesh V. **Algoritmos**. São Paulo: McGraw-Hill, 2009.

GOLOVACH, Petr A. *et al.* **A Survey on the Computational Complexity of Colouring Graphs with Forbidden Subgraphs**. 15 fev. 2016. DOI: 10.48550/arXiv.1407.1482. arXiv: 1407.1482 [cs.CC]. Disponível em: <http://arxiv.org/abs/1407.1482>. Acesso em: 19 jun. 2026. Pré-publicado.

LEWIS, Rhyd. **Graph Colouring: A Visual Tour**. 20 fev. 2026. DOI: 10.48550/arXiv.2602.18246. arXiv: 2602.18246 [math.HO]. Disponível em: <http://arxiv.org/abs/2602.18246>. Acesso em: 19 jun. 2026. Pré-publicado.

REIS, Dalton Solano dos. **Implementação do Trabalho Final da disciplina de Projeto e Análise de Algoritmos**. GitHub. 2026. Disponível em: https://github.com/dalton-reis/UDESC_AlunoEspecial_2026-1_ProjetoAnaliseAlgoritmos/blob/main/_TrabalhoFinal/main.ipynb. Acesso em: 20 jun. 2026.

SILVA, André Tavares da. Projeto e análise de algoritmos: Estudo da tratabilidade de problemas computacionais. [S. l.], 2026. Disponível em: https://moodle.joinville.udesc.br/pluginfile.php/449889/mod_resource/content/2/PAA_12.pdf.

SILVA, André Tavares da. Projeto e análise de algoritmos: Reduções de problemas. [S. l.], 2026. Disponível em: https://moodle.joinville.udesc.br/pluginfile.php/449893/mod_resource/content/2/PAA_13.pdf.